10 언어 모델이란?

"이 다음에 올 단어로 무엇이 가장 자연스러운가"를 확률로 계산하는 모델이 언어 모델입니다. 신경망 이전의 가장 단순한 방법인 "세어 보기"로 원리를 세우고, 그 한계와 평가 방법까지 숫자로 확인합니다.

오늘 다루는 것

1. 언어 모델이란?: 문장의 확률, 다음 단어 예측
2. 통계적 언어 모델과 N-gram 언어 모델: 연쇄 법칙, 카운트, 희소 문제
3. 펄플렉서티(PPL): 언어 모델의 평가

끝나고 나면 할 수 있는 것

1. 언어 모델을 정의하고, 문장의 확률을 조건부 확률의 곱으로 쓸 수 있다.
2. 작은 말뭉치에서 bigram 확률과 문장 확률을 직접 계산하고, 희소 문제가 왜 생기는지 설명할 수 있다.
3. 펄플렉서티를 계산하고, 07장에서 본 손실과 어떤 관계인지 설명할 수 있다.

2026년 10월 8일 목요일 14:30 (10/8 자료 2/2)
발표자 설명, 코랩 실습, 퀴즈 돌아가며 풀기 순서로 진행합니다. 이 자료의 실습은 추가 설치 없이 파이썬 기본 기능만으로 실행됩니다.

10

언어 모델이란?

휴대폰 키보드의 다음 단어 추천, 검색창의 자동 완성, 번역기, 음성 인식, 그리고 ChatGPT. 모두 "지금까지의 단어들 다음에 무엇이 올까"를 계산합니다. 단어의 나열에 확률을 매기는 모델이 언어 모델입니다. 07-03의 Char RNN이 다음 글자를 맞혔던 것을 단어 단위로 옮긴 것이기도 합니다. 10장에서는 신경망을 쓰지 않고 횟수를 세는 방법으로 언어 모델을 만들어 보고, 그 방법이 어디서 막히는지 확인합니다.

왜 배우나: 오늘의 학습 경로



개념마다 먼저 정의와 식을 세우고, 작은 말뭉치에 대입해 직접 계산합니다. 세어서 만드는 언어 모델의 한계(희소 문제)를 숫자로 보고 나면, 왜 RNN 언어 모델과 GPT 같은 신경망 언어 모델이 필요한지 분명해집니다.

10-01 언어 모델이란?

핵심 한 줄

언어 모델은 단어의 나열(문장)에 확률을 매기는 모델이고, 가장 흔한 방식은 앞 단어들을 보고 다음 단어의 확률을 예측하는 것이다.

왜 배우나

문장에 확률을 매길 수 있으면 "어느 문장이 더 자연스러운가"를 숫자로 비교할 수 있습니다. 번역 결과 고르기, 오타 고치기, 음성 인식 후보 고르기가 모두 이 비교입니다. 그리고 "다음 단어 맞히기"를 반복하면 글을 이어 쓸 수 있습니다. 21장의 GPT가 바로 이 일을 하는 거대한 언어 모델입니다.

1) 언어 모델의 정의

정의

- 언어 모델(Language Model, LM): 단어 시퀀스(단어의 나열)에 확률을 할당하는 모델. 그 문장이 얼마나 자연스러운지를 0과 1 사이의 숫자로 나타냅니다.
- 단어 시퀀스의 확률: 단어 하나를 w, 단어 n개로 이루어진 시퀀스를 W라고 하면

$$P(W) = P(w_1, w_2, w_3, \ldots, w_n)$$

말로 하면: "w_1, w_2, …, w_n이 이 순서로 나올 확률". 쉼표는 "그리고"입니다.

- 언어 모델을 만드는 방법은 두 가지입니다. 말뭉치에서 횟수를 세는 **통계적 언어 모델**(10-02)과, 신경망으로 학습하는 **신경망 언어 모델**(RNN, GPT 등)입니다.

예시 적용

확률을 매기면 두 문장 중 더 자연스러운 쪽을 고를 수 있습니다. 왼쪽이 오른쪽보다 확률이 높아야 좋은 언어 모델입니다.

쓰임	확률이 높아야 하는 문장	확률이 낮아야 하는 문장
기계 번역	나는 버스를 탔다	나는 버스를 태운다
오타 교정	선생님이 교실로 부리나케 달려갔다	선생님이 교실로 부리나케 잘려갔다
음성 인식	나는 메론을 먹는다	나는 메롱을 먹는다

음성 인식에서 "메론"과 "메롱"은 소리가 비슷해 둘 다 후보가 됩니다. 소리만으로는 가릴 수 없고, 언어 모델이 "먹는다" 앞에는 "메론을"이 훨씬 자연스럽다고 판단해 줍니다.

2) 다음 단어 예측과 조건부 확률

정의

- 조건부 확률 $P(B \mid A)$: A가 일어났을 때 B가 일어날 확률. 세로줄 $\mid$는 "~이 주어졌을 때"로 읽습니다.
- 다음 단어 예측: 앞의 $n-1$개 단어가 주어졌을 때 n번째 단어의 확률을 구하는 것

$$P(w_n \mid w_1, w_2, \ldots, w_{n-1})$$

말로 하면: 앞 단어들이 $w_1, \ldots, w_{n-1}$이었을 때 다음 단어가 w_n일 확률. 가능한 모든 다음 단어에 대해 이 값을 구하면 합이 1인 확률 분포가 됩니다.

- 문장 전체의 확률은 단어를 하나씩 예측한 확률을 모두 곱한 것입니다(식의 유도는 10-02).

$$P(W) = P(w_1, w_2, \ldots, w_n) = \prod_{i=1}^{n} P(w_i \mid w_1, \ldots, w_{i-1})$$

$\prod$(파이)는 "모두 곱하기"입니다. 그래서 "문장에 확률을 매기는 일"과 "다음 단어를 맞히는 일"은 같은 일입니다. 다음 단어의 확률을 구할 수 있으면 문장의 확률도 구할 수 있습니다.

예시 적용

"비행기를 타려고 공항에 갔는데 지각을 하는 바람에 비행기를 ___"에서 사람은 "놓쳤다"를 떠올립니다. 앞에 나온 "지각"이라는 단서 덕분입니다. 기계는 많은 글을 보고 이 직관을 확률로 배웁니다. 문장 4개짜리 아주 작은 말뭉치에서 "비행기를" 다음에 온 단어를 세어 봤습니다(실습 1).

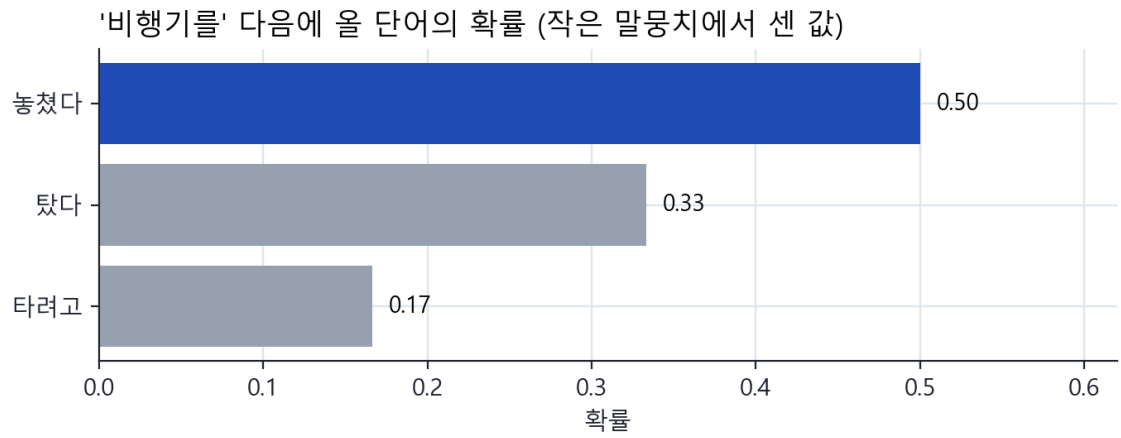


그림 1 "비행기를"은 6번 나왔고, 그 뒤에 "놓쳤다" 3번, "탔다" 2번, "타려고" 1번이 왔다. 세 확률의 합은 1이다

이 모델은 "비행기를" 다음 단어로 확률 0.50인 "놓쳤다"를 추천합니다. 검색창에 "딥 러닝을 이용한"까지 치면 "자연어 처리"가 추천되는 것도 같은 원리입니다.

비유

말끝을 흐린 친구

친구가 "오늘 점심은 김치…"라고 말을 멈추면 우리는 "찌개"나 "볶음밥"을 떠올리지 "자동차"는 떠올리지 않습니다. 평소에 많이 들어 본 표현일수록 머릿속 확률이 높습니다. 언어 모델은 이 눈치를 수많은 글에서 배운 것입니다.

지금까지 들은 말 = = 앞 단어들 (문맥)
떠오르는 다음 말 후보 = = 다음 단어의 확률 분포
평소에 들은 말들 = = 말뭉치 (학습 데이터)

실습 1

작은 말뭉치에서 다음 단어 확률 세기

```python
from collections import Counter

# 아주 작은 "말뭉치": 사람들이 실제로 쓴 문장들
corpus = ["비행기를 타려고 공항에 갔는데 지각을 하는 바람에 비행기를 놓쳤다",
          "늦잠을 자서 비행기를 놓쳤다",
          "공항에 일찍 가서 비행기를 탔다",
          "비행기를 놓쳤다 다음 비행기를 탔다"]

# "비행기를" 다음에 어떤 단어가 왔는지 세어 보기
next_words = Counter()
for sentence in corpus:
    words = sentence.split()
    for prev, nxt in zip(words, words[1:]):
        if prev == "비행기를":
            next_words[nxt] += 1

total = sum(next_words.values())
print("'비행기를' 다음 단어의 확률")
for w, c in next_words.most_common():
    print(f"  {w:5s}: {c}/{total} = {c / total:.2f}")
print("가장 그럴듯한 다음 단어:", next_words.most_common(1)[0][0])
```

실행 결과

```
'비행기를' 다음 단어의 확률
  놓쳤다  : 3/6 = 0.50
  탔다   : 2/6 = 0.33
  타려고  : 1/6 = 0.17
가장 그럴듯한 다음 단어: 놓쳤다
```

직접 해 보기 말뭉치에 "공항에서 비행기를 탔다"를 두 번 더 추가하면 1등 단어가 바뀌나요? 언어 모델이 학습한 글에 따라 달라진다는 것을 확인해 보세요.

10-01 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 언어 모델은 단어 시퀀스에 확률을 할당하는 모델이다. (O / X) 이름 _____

2 언어 모델은 번역이나 음성 인식에는 쓰이지 않는다. (O / X) 이름 _____

3 $P(B \mid A)$를 바르게 읽은 것은? 이름 _____

① A와 B가 함께 일어날 확률 ② A가 일어났을 때 B가 일어날 확률 ③ B가 일어났을 때 A가 일어날 확률

4 음성 인식에서 언어 모델이 하는 일은? 이름 _____

① 소리를 키운다 ② 소리가 비슷한 후보 중 더 자연스러운 문장을 고른다 ③ 불용어를 지운다

5 문장 전체의 확률은 각 단어의 조건부 확률을 차례로 _____ 것이다. 이름 _____

6 실습에서 "비행기를" 다음에 "놓쳤다"가 올 확률은 _____였다. 이름 _____

7 언어 모델을 정의하고, "메론/메롱" 예시로 어디에 쓰이는지 설명해 보세요. 이름 _____

원문: 10-01 언어 모델이란? — https://wikidocs.net/217247

10-02 통계적 언어 모델과 N-gram 언어 모델

핵심 한 줄 **통계적 언어 모델은 말뭉치에서 횟수를 세어 확률을 구하고, N-gram은 문맥을 바로 앞 N-1개 단어로 줄여서 셀 수 있는 경우를 늘린다.**

왜 배우나 10-01의 조건부 확률을 실제로 어떻게 구할까요? 가장 단순한 답은 "세어 보기"입니다. 세는 방식이 왜 금방 한계에 부딪히는지(희소 문제), N-gram이 그것을 어떻게 덜어 주고 무엇을 잃는지 알면 신경망 언어 모델이 등장한 이유가 분명해집니다.

1) 조건부 확률의 연쇄 법칙과 문장의 확률

정의

조건부 확률의 정의에서 출발합니다.

$$P(B \mid A) = \frac{P(A, B)}{P(A)} \quad \Longrightarrow \quad P(A, B) = P(A)\,P(B \mid A)$$

말로 하면: A와 B가 함께 일어날 확률 = A가 일어날 확률 × (A가 일어났을 때 B가 일어날 확률).

이것을 여러 개로 늘린 것이 연쇄 법칙(chain rule)입니다.

$$P(x_1, x_2, \ldots, x_n) = P(x_1)\,P(x_2 \mid x_1)\,P(x_3 \mid x_1, x_2) \cdots P(x_n \mid x_1, \ldots, x_{n-1})$$

첫 번째가 일어날 확률, 첫 번째가 일어났을 때 두 번째가 일어날 확률, 앞의 둘이 일어났을 때 세 번째가 일어날 확률, …을 차례로 곱합니다. x를 단어 w로 바꾸면 10-01의 문장 확률 식이 됩니다.

예시 적용 문장 "An adorable little boy is spreading smiles"(사랑스러운 작은 소년이 미소를 퍼뜨리고 있다)의 확률은 단어 7개에 대한 확률 7개의 곱입니다.

순서	곱하는 확률	뜻
1	$P(\text{An})$	문장이 An으로 시작할 확률
2	$P(\text{adorable} \mid \text{An})$	An 다음에 adorable이 올 확률
3	$P(\text{little} \mid \text{An adorable})$	An adorable 다음에 little이 올 확률
…	…	…
7	$P(\text{smiles} \mid \text{An adorable little boy is spreading})$	앞의 여섯 단어 다음에 smiles가 올 확률

뒤로 갈수록 조건(세로줄 뒤의 문맥)이 길어진다는 점을 기억해 두세요. 이것이 곧 문제가 됩니다.

2) 카운트 기반 접근과 희소 문제

정의

- 통계적 언어 모델(Statistical Language Model, SLM): 말뭉치에서 단어 시퀀스가 나온 **횟수(count)**로 확률을 구하는 언어 모델

$$P(w_n \mid w_1, \ldots, w_{n-1}) = \frac{\text{count}(w_1, \ldots, w_{n-1}, w_n)}{\text{count}(w_1, \ldots, w_{n-1})}$$

말로 하면: 문맥 $w_1, \ldots, w_{n-1}$이 나온 횟수 중에서, 그 뒤에 w_n이 이어진 횟수의 비율.

- 희소 문제(sparsity problem): 충분한 데이터를 보지 못해 확률을 제대로 구하지 못하는 문제
  - 분자가 0인 경우: 그 단어 시퀀스가 말뭉치에 없으면 확률이 0이 됩니다. 실제로는 충분히 가능한 문장인데도 "절대 일어나지 않는다"고 답하게 됩니다.
  - 분모가 0인 경우: 문맥 자체가 말뭉치에 없으면 확률을 정의할 수 없습니다.

왜 피하기 어려울까? 문맥이 길수록 정확히 같은 단어 나열이 말뭉치에 있을 가능성은 급격히 줄어듭니다. 단어가 1만 종류라면 단어 5개의 나열은 $10000^5 = 10^{20}$가지나 됩니다. 아무리 큰 말뭉치도 그중 극히 일부만 담고 있습니다.

예시 적용 말뭉치에 "An adorable little boy"가 100번 나왔고 그 뒤에 "is"가 30번 왔다면 $P(\text{is} \mid \text{An adorable little boy}) = 30/100 = 0.3$입니다.

그런데 말뭉치에 "An adorable little boy"가 한 번도 없었다면 분모가 0이라 이 확률을 구할 수 없습니다. "An adorable little boy is"가 없었다면 확률이 0입니다. 둘 다 사람이 보기에 자연스러운 표현인데도 그렇습니다.

3) N-gram 언어 모델

정의

- N-gram: 연속한 N개 단어의 묶음. $N = 1$이면 유니그램(unigram), 2이면 바이그램(bigram), 3이면 트라이그램(trigram), 4 이상은 4-gram, 5-gram으로 부릅니다.
- N-gram 언어 모델: 다음 단어를 예측할 때 앞의 모든 단어가 아니라 **바로 앞 $N-1$개 단어만** 보는 통계적 언어 모델

$$P(w_n \mid w_1, \ldots, w_{n-1}) \approx P(w_n \mid w_{n-N+1}, \ldots, w_{n-1}) = \frac{\text{count}(w_{n-N+1}, \ldots, w_{n-1}, w_n)}{\text{count}(w_{n-N+1}, \ldots, w_{n-1})}$$

$\approx$는 "거의 같다고 치자"입니다. 긴 문맥 대신 짧은 문맥으로 바꿔 세면, 짧은 단어 묶음은 말뭉치에 있을 가능성이 훨씬 높으므로 셀 수 있게 됩니다. bigram($N = 2$)이면 $P(w_n \mid w_{n-1}) = \text{count}(w_{n-1}, w_n)/\text{count}(w_{n-1})$입니다.

모델	보는 문맥	"… boy is spreading ___"에서 쓰는 확률
unigram ($N = 1$)	없음 (단어 자체의 빈도만)	$P(w)$
bigram ($N = 2$)	앞 1단어	$P(w \mid \text{spreading})$
trigram ($N = 3$)	앞 2단어	$P(w \mid \text{is spreading})$
4-gram ($N = 4$)	앞 3단어	$P(w \mid \text{boy is spreading})$

N-gram의 한계와 N을 고르는 문제.

- **희소 문제는 줄어들 뿐 없어지지 않습니다.** 짧은 묶음도 말뭉치에 없을 수 있습니다.
- **앞 문맥을 잊습니다.** 문맥 밖의 단어는 아예 보지 않으므로, 문장 앞부분과 뒷부분이 어긋난 예측을 하기도 합니다.
- **N은 트레이드오프입니다.** N이 크면 문맥을 많이 봐서 정확하지만 셀 수 있는 경우가 줄고(희소 문제), 모든 N-gram의 횟수를 저장해야 해서 모델도 커집니다. N이 작으면 잘 세지지만 실제 언어와 멀어집니다. 그래서 보통 N은 5를 넘기지 않습니다.

예시 적용 문장 네 개짜리 말뭉치로 bigram 모델을 만듭니다(실습 2). 문장의 시작과 끝도 맞혀야 하므로 앞뒤에 `<s>`, `</s>` 토큰을 붙입니다.

an adorable little boy is spreading smiles / an adorable little boy is sleeping / a little boy is spreading smiles / the boy is spreading joy

"is"는 4번 나왔고 그 뒤에 "spreading"이 3번, "sleeping"이 1번 왔으므로 $P(\text{spreading} \mid \text{is}) = 3/4 = 0.75$, $P(\text{sleeping} \mid \text{is}) = 1/4 = 0.25$입니다. 이제 말뭉치에 없는 새 문장 두 개의 확률을 구해 봅니다.

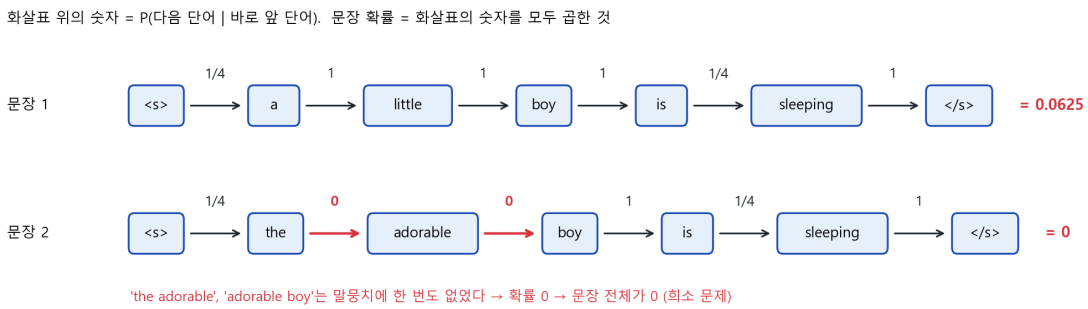


그림 2 bigram 모델로 구한 문장 확률. 문장 1은 처음 보는 문장인데도 확률이 나온다. 문장 2는 본 적 없는 단어 쌍이 있어 0이 된다

- **문장 1 "a little boy is sleeping"**: 말뭉치에 이 문장은 없습니다. 하지만 "a little", "little boy", "boy is", "is sleeping"은 모두 본 적이 있어서 $\frac{1}{4} \times 1 \times 1 \times 1 \times \frac{1}{4} \times 1 = 0.0625$가 나옵니다. 문맥을 줄인 덕분에 **처음 보는 문장에도 확률을 매길 수 있게** 됐습니다.
- **문장 2 "the adorable boy is sleeping"**: "the adorable"과 "adorable boy"를 본 적이 없어 그 자리의 확률이 0이고, 곱이므로 문장 전체가 0이 됩니다. 희소 문제가 남아 있습니다.



그림 3 "spreading" 다음 단어를 예측할 때 실제로 보는 범위. 줄 그은 단어는 무시된다. bigram은 "boy"가 주어인지도 모른다

N이 커지면 얼마나 희소해질까? 파이썬에 들어 있는 글 "The Zen of Python" 19줄을 앞 14줄(학습)과 뒤 5줄(시험)로 나눠, 시험 글의 N-gram 가운데 학습 글에서 본 적 있는 것의 비율을 재 봤습니다(실습 3).

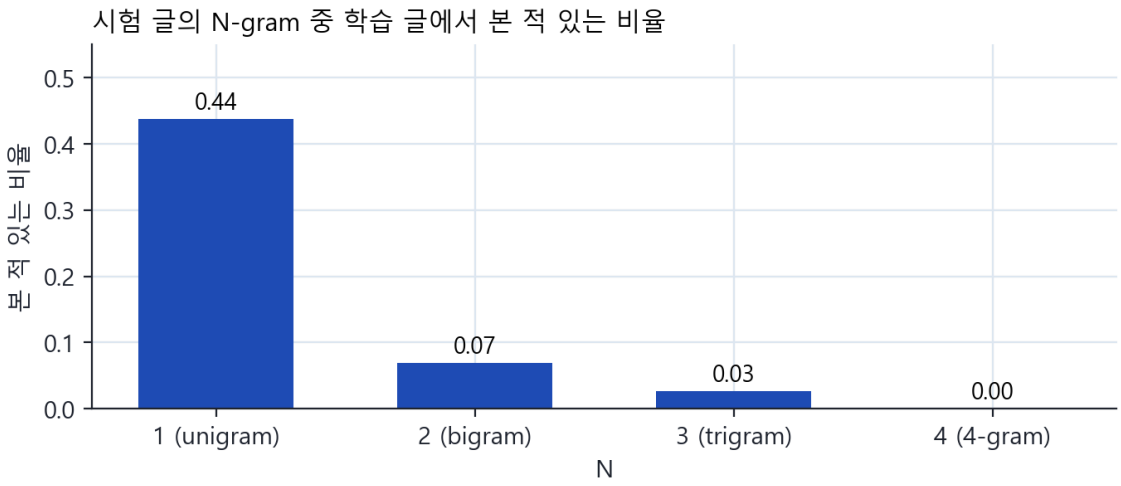


그림 4 단어 하나는 44%를 본 적이 있지만, 두 단어 묶음은 7%, 세 단어 묶음은 3%, 네 단어 묶음은 하나도 없다

본 적 있는 trigram은 "is better than" 하나뿐이었습니다. N을 키우면 본 적 없는 묶음이 대부분이 되어 확률이 0투성이가 됩니다. 반대로 큰 말뭉치에서는 N을 키울수록 성능이 좋아집니다. 교재가 소개하는 월스트리트 저널 실험(학습 3,800만 단어)에서 펄플렉서티(10-03, 낮을수록 좋음)는 unigram 962, bigram 170, trigram 109였습니다.

비유

기억력이 짧은 친구

바로 앞 한두 마디만 기억하는 친구는 대화가 길어지면 "아까 무슨 얘기였지?" 하며 엉뚱한 대답을 합니다. 그래도 짧은 표현("고마워" 다음엔 "천만에")은 잘 맞힙니다. 반대로 들은 대화를 통째로 외워서만 대답하는 친구는 처음 듣는 문장 앞에서는 아무 대답도 못 합니다.

앞 한두 마디만 기억 = = N-gram의 N-1개 문맥
통째로 외운 대화만 대답 = = 전체 문맥으로 세는 모델
처음 듣는 문장에 얼어붙음 = = 희소 문제 (확률 0)

실습 2 bigram 언어 모델로 문장 확률 구하기

```python
from collections import Counter

corpus = ["an adorable little boy is spreading smiles",
          "an adorable little boy is sleeping",
          "a little boy is spreading smiles",
          "the boy is spreading joy"]
# 문장의 시작 <s>와 끝 </s>도 토큰으로 넣는다
sents = [["<s>"] + s.split() + ["</s>"] for s in corpus]

def ngrams(words, n):
    """연속한 n개 단어 묶음을 모두 만든다."""
    return [tuple(words[i:i + n]) for i in range(len(words) - n + 1)]

words = "an adorable little boy".split()
print("bigram :", ngrams(words, 2))
print("trigram:", ngrams(words, 3))

uni = Counter(w for s in sents for w in s)              # 단어 1개의 횟수
bi = Counter(g for s in sents for g in ngrams(s, 2))    # 단어 2개 묶음의 횟수

def p(word, prev):
    """bigram 확률 P(word | prev) = count(prev word) / count(prev)"""
    return bi[(prev, word)] / uni[prev]

print("\ncount(is) =", uni["is"], " count(is spreading) =", bi[("is", "spreading")])
print("P(spreading | is) =", p("spreading", "is"))
print("P(sleeping  | is) =", p("sleeping", "is"))

def sentence_prob(sentence):
    """문장 확률 = 단어마다의 조건부 확률을 차례로 곱한 것"""
    tokens = ["<s>"] + sentence.split() + ["</s>"]
    prob = 1.0
    for prev, w in zip(tokens, tokens[1:]):
        q = p(w, prev)
        print(f"  P({w:8s}| {prev:9s}) = {bi[(prev, w)]}/{uni[prev]} = {q:.2f}")
        prob *= p(w, prev)
    return prob

print("\n문장 1: a little boy is sleeping")
print("  문장 확률 =", sentence_prob("a little boy is sleeping"))
print("\n문장 2: the adorable boy is sleeping")
print("  문장 확률 =", sentence_prob("the adorable boy is sleeping"))
```

실행 결과

```
bigram : [('an', 'adorable'), ('adorable', 'little'), ('little', 'boy')]
trigram: [('an', 'adorable', 'little'), ('adorable', 'little', 'boy')]

count(is) = 4  count(is spreading) = 3
P(spreading | is) = 0.75
P(sleeping  | is) = 0.25

문장 1: a little boy is sleeping
  P(a       | <s>      ) = 1/4 = 0.25
  P(little  | a        ) = 1/1 = 1.00
  P(boy     | little   ) = 3/3 = 1.00
  P(is      | boy      ) = 4/4 = 1.00
  P(sleeping| is       ) = 1/4 = 0.25
  P(</s>    | sleeping ) = 1/1 = 1.00
  문장 확률 = 0.0625

문장 2: the adorable boy is sleeping
  P(the     | <s>      ) = 1/4 = 0.25
  P(adorable| the      ) = 0/1 = 0.00
  P(boy     | adorable ) = 0/2 = 0.00
  P(is      | boy      ) = 4/4 = 1.00
  P(sleeping| is       ) = 1/4 = 0.25
  P(</s>    | sleeping ) = 1/1 = 1.00
  문장 확률 = 0.0
```

직접 해 보기 말뭉치에 "the adorable boy is sleeping"을 추가하면 문장 2의 확률이 0에서 얼마로 바뀌나요? 데이터를 늘리는 것만으로 희소 문제를 완전히 풀 수 있을까요?

실습 3 N이 커질수록 본 적 없는 N-gram이 늘어난다

```
import codecs
import contextlib
import io
import re

# 파이썬에 들어 있는 글 "The Zen of Python" (설치 없이 쓸 수 있는 실제 글)
with contextlib.redirect_stdout(io.StringIO()):   # import할 때 나오는 출력을 숨김
    import this
# this.s는 암호(ROT13)로 저장되어 있어서 풀어 준다
text = codecs.decode(this.s, "rot13").lower()
lines = [re.findall(r"[a-z']+", line) for line in text.split("\n")[2:]]
lines = [l for l in lines if l]
train, test = lines[:14], lines[14:]            # 앞 14줄로 세고, 뒤 5줄로 시험
print("학습:", len(train), "줄,", sum(map(len, train)), "단어,",
      "시험:", len(test), "줄")

def ngrams(words, n):
    return [tuple(words[i:i + n]) for i in range(len(words) - n + 1)]

print("\n N | 학습에서 본 종류 | 시험의 N-gram | 그중 학습에서 본 것 | 비율")
for n in [1, 2, 3, 4]:
    seen = {g for l in train for g in ngrams(l, n)}      # 학습 글에 나온 N-gram
    test_grams = [g for l in test for g in ngrams(l, n)]
    hit = sum(g in seen for g in test_grams)
    ratio = hit / len(test_grams)
    print(f" {n} | {len(seen):14d} | {len(test_grams):11d} | {hit:17d} | {ratio:.2f}")

print("\n시험 글의 trigram 중 학습에서 본 것:")
seen3 = {g for l in train for g in ngrams(l, 3)}
print(" ", [" ".join(g) for l in test for g in ngrams(l, 3) if g in seen3])
```

실행 결과

```
학습: 14 줄, 88 단어, 시험: 5 줄

 N | 학습에서 본 종류 | 시험의 N-gram | 그중 학습에서 본 것 | 비율
 1 |             60 |          48 |                21 | 0.44
 2 |             64 |          43 |                 3 | 0.07
 3 |             55 |          38 |                 1 | 0.03
 4 |             47 |          33 |                 0 | 0.00

시험 글의 trigram 중 학습에서 본 것:
  ['is better than']
```

직접 해 보기 `train, test = lines[:14], lines[14:]` 의 14를 17로 바꿔 학습 글을 늘리면 비율이 어떻게 달라지나요?

확인 문제

10-02 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 통계적 언어 모델은 말뭉치에서 단어 시퀀스가 나온 횟수를 세어 확률을 구한다. (O / X) 이름 _____

2 N을 크게 할수록 희소 문제가 줄어든다. (O / X) 이름 _____

3 단어 2개 묶음을 부르는 이름은? 이름 _____

① unigram ② bigram ③ trigram

4 말뭉치에 한 번도 나오지 않은 단어 시퀀스 때문에 확률이 0이 되는 문제는? 이름 _____

① 과적합 ② 희소 문제 ③ 기울기 소실

5 4-gram은 다음 단어를 예측할 때 바로 앞의 _____개 단어만 본다. 이름 _____

6 실습에서 $P(\text{spreading} \mid \text{is})$는 _____였다. 이름 _____

7 N-gram 언어 모델을 정의하고, 실습의 문장 1과 문장 2로 장점과 한계를 설명해 보세요. 이름 _____

원문: 10-02 통계적 언어 모델과 N-gram 언어 모델 — https://wikidocs.net/217248 ("The Zen of Python"으로 희소성을 재는 실습 3은 이 자료에서 추가한 실험입니다)

10-03 펄플렉서티(PPL)

핵심 한 줄 **펄플렉서티는 언어 모델이 다음 단어를 고를 때 평균 몇 개 중에서 헷갈리는지를 나타내는 숫자이고, 낮을수록 좋다.**

왜 배우나 언어 모델 두 개 중 어느 쪽이 나은지 비교하려면 점수가 필요합니다. 번역 같은 실제 일에 넣어 보는 것이 가장 정확하지만 시간이 오래 걸립니다. 펄플렉서티는 시험용 문장만으로 빠르게 계산할 수 있는 대표적인 평가 지표이고, 신경망 언어 모델의 논문에서도 그대로 쓰입니다.

1) 펄플렉서티의 정의

정의

펄플렉서티(perplexity, PPL)는 언어 모델의 평가 지표입니다. perplexed는 "헷갈리는"이라는 뜻이고, PPL은 모델이 시험 문장을 보고 헷갈리는 정도입니다. 문장 W의 단어 수가 N일 때

$$PPL(W) = P(w_1, w_2, \dots, w_N)^{-\frac{1}{N}} = \sqrt[N]{\frac{1}{P(w_1, w_2, \dots, w_N)}}$$

말로 하면: 모델이 시험 문장에 준 확률을 뒤집고(역수), 단어 수만큼 제곱근을 씌운 값. 모델이 실제 문장에 확률을 높게 줄수록 PPL은 작아집니다. 그래서 **낮을수록 좋습니다**.

문장 확률에 연쇄 법칙을 쓰면 단어별 확률의 곱으로 바뀌고, bigram 모델이라면 문맥이 앞 한 단어가 됩니다.

$$PPL(W) = \sqrt[N]{\frac{1}{\prod_{i=1}^{N} P(w_i \mid w_1, \dots, w_{i-1})}} \qquad PPL_{\text{bigram}}(W) = \sqrt[N]{\frac{1}{\prod_{i=1}^{N} P(w_i \mid w_{i-1})}}$$

왜 N제곱근을 씌울까? 문장 확률은 1보다 작은 수를 계속 곱한 것이라, 문장이 길수록 무조건 작아집니다. 그대로 비교하면 긴 문장이 늘 불리합니다. N제곱근은 곱의 평균(기하평균)을 내는 것이고, "단어 하나당 평균적으로 얼마의 확률을 줬는가"로 바꿔 줍니다. 그래서 길이가 다른 문장도 비교할 수 있습니다.

07장에서 본 손실과의 관계. 실제 코드에서는 확률을 곱하지 않고 로그를 더합니다(아주 작은 수를 계속 곱하면 컴퓨터가 0으로 처리해 버리기 때문).

$$PPL(W) = \exp\left(-\frac{1}{N}\sum_{i=1}^{N} \log P(w_i \mid w_1, \dots, w_{i-1})\right) = \exp(\text{loss})$$

괄호 안은 "정답 단어에 준 확률의 로그에 마이너스를 붙여 평균 낸 것", 곧 크로스 엔트로피 손실입니다(05장, 07-03). 그래서 **PPL은 손실에 exp를 씌운 값**입니다. 신경망 언어 모델은 손실을 줄이도록 학습하므로, 학습이 곧 PPL을 낮추는 일입니다.

예시 적용 10-02의 bigram 모델로 세 문장의 PPL을 구합니다(실습 5). N은 맞혀야 하는 단어의 수이고, 문장 끝 `</s>` 도 맞혀야 하므로 포함합니다.

시험 문장	단어별 확률	문장 확률	N	PPL
an adorable little boy is spreading smiles	0.5, 1, 1, 1, 1, 0.75, 0.67, 1	0.25	8	$0.25^{-1/8} = 1.19$
a little boy is sleeping	0.25, 1, 1, 1, 0.25, 1	0.0625	6	$0.0625^{-1/6} = 1.59$
the adorable boy is sleeping	0.25, 0, 0, 1, 0.25, 1	0	6	무한대

말뭉치에 그대로 있던 첫 문장이 가장 덜 헷갈리고(1.19), 처음 보는 둘째 문장은 조금 더 헷갈립니다(1.59). 셋째 문장은 확률 0인 자리가 있어 PPL이 무한대입니다. 희소 문제가 있으면 평가조차 할 수 없다는 뜻이고, 실제 N-gram 모델이 0을 피하는 보정 기법을 함께 쓰는 이유입니다.

손실과의 관계도 숫자로 확인합니다. 07-03 심화 실습(문자 25종류)의 손실을 PPL로 바꾸면 이렇습니다.

상태	손실	PPL = exp(손실)	뜻
25개 중 아무거나 찍기	$\ln 25 = 3.219$	25.00	매번 25개 중에서 고민
학습 전	3.297	27.03	찍는 것과 비슷함
학습 후	0.278	1.32	거의 한 글자로 좁힘

2) 분기 계수

정의

- 분기 계수(branching factor): 매 시점에 고를 수 있는 선택지의 수. PPL은 "이 모델이 매 시점 평균 몇 개의 후보를 놓고 고민하는가"로 읽을 수 있습니다.

$$P(w_i \mid \cdot) = \frac{1}{k} \text{ for all } i \quad \Longrightarrow \quad PPL(W) = \left(\left(\tfrac{1}{k}\right)^N\right)^{-\frac{1}{N}} = k$$

매 단어마다 정답에 $1/k$의 확률을 주는 모델, 곧 k개 후보를 똑같이 놓고 고민하는 모델의 PPL은 정확히 k입니다. 문장 길이 N은 사라집니다.

PPL을 볼 때 주의할 점.

- PPL이 낮다는 것은 그 시험 데이터에서 정답 단어에 높은 확률을 줬다는 뜻이지, 사람이 느끼기에 좋은 글을 쓴다는 보장은 아닙니다.
- PPL은 시험 데이터에 따라 달라집니다. 두 모델을 비교할 때는 **같은 시험 데이터**(충분히 크고, 쓰려는 분야의 글)로 재야 합니다.

예시 적용

매 단어마다 정답에 0.5를 주는 모델 A의 PPL은 2, 0.1을 주는 모델 B의 PPL은 10입니다(실습 4). 모델 A는 2개 중에서, 모델 B는 10개 중에서 고민하는 수준입니다. 모델 A에서 문장 길이를 6, 12, 24로 늘리면 문장 확률은 0.016, 0.00024, 0.00000006으로 급격히 줄지만 PPL은 계속 2입니다. N제곱근이 길이의 영향을 없애 준 것입니다.

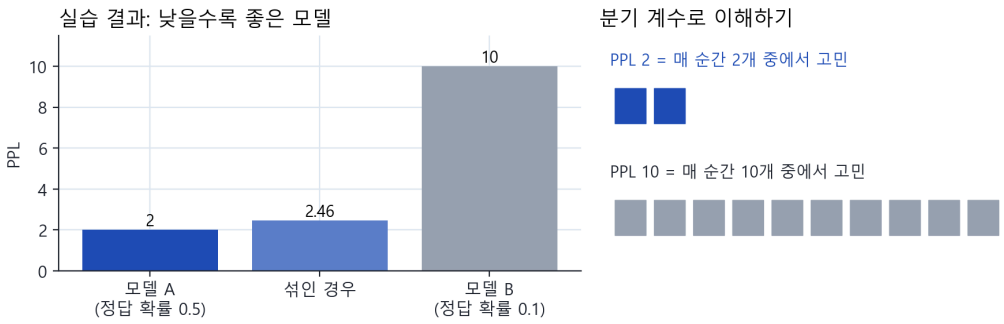


그림 5 왼쪽: 실습 4의 세 경우. 오른쪽: PPL을 "매 순간 고민하는 후보 수"로 읽기

단어마다 확률이 0.9, 0.6, 0.3, 0.8, 0.05, 0.7로 다른 경우의 PPL은 2.46입니다. 다섯 단어를 잘 맞혀도 0.05짜리 하나가 PPL을 끌어올립니다.

교재가 소개하는 페이스북 AI 연구팀의 실험에서, 보정 기법을 쓴 5-gram 언어 모델의 PPL은 67.6이었고 신경망 언어 모델들은 그보다 낮은 값을 기록했습니다. 세어서 만드는 모델보다 신경망이 다음 단어를 더 잘 맞힌다는 뜻이고, 이 흐름이 16장 이후의 신경망 모델로 이어집니다.

비유

객관식 시험의 보기 개수

같은 문제라도 보기가 2개면 찍어도 반은 맞히고, 보기가 10개면 훨씬 어렵습니다. 실력 있는 학생은 보기가 많아도 마음속에서 후보를 2~3개로 좁힙니다. PPL은 모델이 "마음속으로 몇 개 보기 중에서 고르고 있는가"입니다.

마음속 보기 개수 = = 펄플렉서티 (분기 계수)

보기를 잘 좁히는 학생 = = 좋은 언어 모델 (낮은 PPL)

같은 시험지로 비교 = = 같은 시험 데이터로 비교

실습 4 펄플렉서티 계산하기

```python
import math

def perplexity(word_probs):
    """단어마다 모델이 정답에 준 확률을 받아 PPL을 구한다."""
    n = len(word_probs)
    sentence_prob = math.prod(word_probs)        # 문장 확률 = 모두 곱한 것
    return sentence_prob ** (-1 / n)             # -1/N 제곱 (N제곱근의 역수)

# 모델 A: 매 단어마다 정답에 확률 0.5 / 모델 B: 매 단어마다 0.1
print("모델 A PPL:", round(perplexity([0.5] * 6), 2))
print("모델 B PPL:", round(perplexity([0.1] * 6), 2))

# 문장이 길어져도 PPL은 그대로다 (문장 확률은 계속 작아진다)
for n in [6, 12, 24]:
    probs = [0.5] * n
    print(f"  길이 {n:2d}: 문장 확률 {math.prod(probs):.8f}, PPL {perplexity(probs):.2f}")

# 실제로는 단어마다 확률이 다르다
mixed = [0.9, 0.6, 0.3, 0.8, 0.05, 0.7]
print("섞인 경우 PPL:", round(perplexity(mixed), 2))

# 같은 값을 로그로 계산: PPL = exp(-(로그 확률의 평균))
avg_nll = -sum(math.log(q) for q in mixed) / len(mixed)
print(f"로그로 계산: 평균 손실 {avg_nll:.3f} → exp = {math.exp(avg_nll):.2f}")
```

실행 결과

```
모델 A PPL: 2.0
모델 B PPL: 10.0
  길이  6: 문장 확률 0.01562500, PPL 2.00
  길이 12: 문장 확률 0.00024414, PPL 2.00
  길이 24: 문장 확률 0.00000006, PPL 2.00
섞인 경우 PPL: 2.46
로그로 계산: 평균 손실 0.899 → exp = 2.46
```

직접 해 보기 `mixed` 의 0.05를 0.5로 바꾸면 PPL이 얼마나 내려가나요? 확률이 아주 낮은 단어 하나가 PPL을 크게 올린다는 점을 확인해 보세요.

실습 5 bigram 모델의 PPL, 그리고 손실과 PPL

```python
import math
from collections import Counter

# 10-02의 말뭉치로 만든 bigram 언어 모델
corpus = ["an adorable little boy is spreading smiles",
          "an adorable little boy is sleeping",
          "a little boy is spreading smiles",
          "the boy is spreading joy"]
sents = [["<s>"] + s.split() + ["</s>"] for s in corpus]
uni = Counter(w for s in sents for w in s)
bi = Counter(pair for s in sents for pair in zip(s, s[1:]))

def ppl(sentence):
    tokens = ["<s>"] + sentence.split() + ["</s>"]
    probs = [bi[(prev, w)] / uni[prev] for prev, w in zip(tokens, tokens[1:])]
    n = len(probs)                               # 맞혀야 하는 단어 수 (</s> 포함)
    if 0 in probs:                               # 확률 0이 하나라도 있으면 무한대
        return probs, math.inf
    return probs, math.prod(probs) ** (-1 / n)   # 문장 확률의 -1/N 제곱

for s in ["an adorable little boy is spreading smiles",
          "a little boy is sleeping",
          "the adorable boy is sleeping"]:
```

```
    probs, value = ppl(s)
    print(s)
    print("  단어별 확률:", [round(q, 2) for q in probs], f"→ PPL = {value:.2f}")

# 손실(크로스 엔트로피)과 PPL: PPL = exp(평균 손실)
print("\n손실 → PPL")
for name, loss in [("25개 중 찍기 (ln 25)", math.log(25)),
                   ("07-03 심화 실습, 학습 전", 3.297),
                   ("07-03 심화 실습, 학습 후", 0.278)]:
    print(f"  {name:22s} 손실 {loss:.3f} → PPL {math.exp(loss):6.2f}")
```

실행 결과

```
an adorable little boy is spreading smiles
  단어별 확률: [0.5, 1.0, 1.0, 1.0, 1.0, 0.75, 0.67, 1.0] → PPL = 1.19
a little boy is sleeping
  단어별 확률: [0.25, 1.0, 1.0, 1.0, 0.25, 1.0] → PPL = 1.59
the adorable boy is sleeping
  단어별 확률: [0.25, 0.0, 0.0, 1.0, 0.25, 1.0] → PPL = inf

손실 → PPL
  25개 중 찍기 (ln 25)        손실 3.219 → PPL  25.00
  07-03 심화 실습, 학습 전      손실 3.297 → PPL  27.03
  07-03 심화 실습, 학습 후      손실 0.278 → PPL   1.32
```

직접 해 보기 시험 문장을 "the boy is sleeping"으로 바꾸면 PPL이 얼마가 될까요? 실습 2의 확률로 먼저 손으로 계산해 보세요.

확인 문제

10-03 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 펄플렉서티는 높을수록 좋은 모델이다. (O / X) 이름 _____

2 PPL이 10이면 매 시점 평균 10개 후보 중에서 고민하는 수준이다. (O / X) 이름 _____

3 매 단어마다 정답에 확률 0.5를 주는 모델의 PPL은? 이름 _____

① 0.5 ② 2 ③ 5 ④ 10

4 PPL 식에서 N제곱근을 씌우는 이유는? 이름 _____

① 문장이 길수록 확률이 작아지는 것을 보정하려고 ② 계산을 빠르게 하려고 ③ 불용어를 무시하려고

5 PPL은 크로스 엔트로피 손실에 _____를 씌운 값이다. 이름 _____

6 실습에서 확률 0인 단어 쌍이 있는 문장의 PPL은 _____였다. 이름 _____

7 펄플렉서티를 정의하고, 분기 계수의 뜻을 객관식 보기 개수에 빗대어 설명해 보세요. 이름 _____

원문: 10-03 펄플렉서티 — https://wikidocs.net/217272 (손실과 PPL의 관계, bigram 모델의 PPL 계산은 이 자료에서 추가한 내용입니다)

오늘의 요약

다시 보기



정의 모음

개념	정의	예시
언어 모델	단어 시퀀스에 확률을 할당하는 모델	"메론을 먹는다" > "메롱을 먹는다"
다음 단어 예측	$P(w_n \mid w_1, \ldots, w_{n-1})$	"비행기를" 다음 "놓쳤다" 0.50
연쇄 법칙	문장 확률 = 단어별 조건부 확률의 곱	7단어 문장 = 확률 7개의 곱
통계적 언어 모델	횟수의 비율로 확률을 구함	30 / 100 = 0.3
희소 문제	말뭉치에서 보지 못해 확률이 0이거나 정의되지 않는 문제	"the adorable" → 문장 확률 0
N-gram 언어 모델	바로 앞 $N-1$개 단어만 보는 통계적 언어 모델	bigram: $P(\text{spreading} \mid \text{is}) = 0.75$
N의 트레이드오프	크면 정확하지만 희소, 작으면 잘 세지지만 문맥 손실	본 적 있는 비율 0.44 → 0.07 → 0.03 → 0
펄플렉서티	$P(W)^{-1/N}$ = exp(손실). 낮을수록 좋음	1.19, 1.59, 무한대 / 손실 0.278 → 1.32
분기 계수	매 시점 평균 몇 개 후보 중에서 고민하는가	정답 확률 0.1 → PPL 10

다음

다음 자료 예고: 11장 텍스트의 유사도, 12장 텍스트 임베딩 (10/12)

지금까지 단어는 번호일 뿐이었습니다. 번호 2(영화)와 3(정말)은 숫자가 가까워도 뜻이 가깝지 않습니다. 다음 시간에는 문서와 단어를 벡터로 바꿔 "얼마나 비슷한가"를 계산하는 방법을 배웁니다. 본 적 없는 단어 묶음에 0을 주던 희소 문제도, 비슷한 단어끼리 비슷한 벡터를 갖게 하면 풀 실마리가 생깁니다.

정답과 해설

10-01 언어 모델이란?

1. O
2. X — 여러 후보 문장 중 더 자연스러운 것을 고르는 데 쓰입니다.
3. ②
4. ②
5. 곱한
6. 0.5 (3/6)
7. 예시 답: 언어 모델은 단어 시퀀스에 확률을 할당하는 모델이다. 음성 인식에서 소리가 비슷한 "메론을 먹는다"와 "메롱을 먹는다" 중 확률이 높은 앞 문장을 고르는 데 쓰인다.

10-02 통계적 언어 모델과 N-gram 언어 모델

1. O
2. X — N이 클수록 정확히 같은 단어 묶음을 찾기 어려워 희소 문제가 커집니다(실습 3: 0.44 → 0.00).
3. ②
4. ②
5. 3
6. 0.75 (3/4)
7. 예시 답: N-gram 언어 모델은 바로 앞 N-1개 단어만 보고 다음 단어의 확률을 세어 구하는 모델이다. bigram은 처음 보는 문장 1에도 확률 0.0625를 줄 수 있었지만(장점), 본 적 없는 단어 쌍 "the adorable"이 있는 문장 2는 0이 됐다(희소 문제). 또 앞 한 단어만 보므로 앞쪽 문맥을 잊는다.

10-03 펄플렉서티

1. X — 낮을수록 좋습니다.
2. O
3. ② — $1/0.5 = 2$
4. ① — 길이 6, 12, 24에서 문장 확률은 줄지만 PPL은 2로 같았습니다.
5. exp (지수 함수) — 손실 0.278 → PPL 1.32
6. 무한대 (inf)
7. 예시 답: 펄플렉서티는 문장 확률의 역수에 단어 수만큼 제곱근을 씌운 값으로, 낮을수록 좋다. 분기 계수는 매 시점 평균 몇 개의 후보 중에서 고민하는지이고, 객관식에서 마음속으로 남긴 보기의 개수와 같다. 보기를 적게 남길수록 좋은 모델이다.

용어 정리

용어	정의
언어 모델 (LM)	단어 시퀀스에 확률을 할당하는 모델
단어 시퀀스	단어를 순서대로 늘어놓은 것 (문장)
조건부 확률 $P(B \mid A)$	A가 일어났을 때 B가 일어날 확률
연쇄 법칙 (chain rule)	여러 사건이 함께 일어날 확률을 조건부 확률의 곱으로 쓰는 법칙
통계적 언어 모델 (SLM)	말뭉치에서 센 횟수로 확률을 구하는 언어 모델
희소 문제	충분한 데이터를 보지 못해 확률을 제대로 구하지 못하는 문제
N-gram	연속한 N개 단어의 묶음 (unigram, bigram, trigram, …)
<s>, </s>	문장의 시작과 끝을 나타내는 특별한 토큰
펄플렉서티 (PPL)	문장 확률의 역수의 N제곱근. 언어 모델의 평가 지표, 낮을수록 좋음
분기 계수	매 시점에 고를 수 있는 선택지의 수